

C73 A2

Changze Wu

September 2024

1 Q1

This pseudo-code implements an algorithm to process a set of video intervals $\mathcal{V}$. It sorts the intervals by decreasing s_i values, processes each interval, and stores the identifier i of each interval into the result set $\mathcal{A}$. Finally, the algorithm returns the result set $\mathcal{A}$.

Algorithm 1 PROCESSVIDEO($\mathcal{V}$)

```
1:  $\mathcal{L} :=$  set of intervals in  $\mathcal{V}$  sorted by decreasing  $s_i$ .
2:  $\mathcal{A} := \emptyset$ 
3:  $a := 0$ 
4: for video in  $\mathcal{L}$  do
5:    $\mathcal{A}[a] :=$  video[1] (which indicate the  $i$  in  $(i, f_i, s_i)$ )
6:    $a++$ 
7: end for
8: return  $\mathcal{A}$ 
```

Correctness: Proof by exchange argument

Let $A = i_1, i_2, \dots, i_n$ be any schedule. The finish time of video i_k in this schedule is $(f_{i_1} + f_{i_2} + \dots + f_{i_k}) + s_{i_k}$.

Then we define an inversion to be two videos i and j such that i appears before j in A , but $s_i < s_j$.

Since the algorithm produces a no-inversion schedule. Any two schedule with no inversions must have same time in second phrase thus they must have same total time. In order to prove the correctness, we only need to prove that the schedule with no inversion is optimal.

Let A^* be an optimal schedule with the minimum number of inversion. WTP: A^* has no inversion. Suppose for contradiction that A^* has at least on inversion. Thus, there must be two videos such as i and j that appear consecutively in A^* but $s_i < s_j$. Let A' be schedule with i and j swapped. We know that apart from i and j , every other videos has same completion in A^* and A' . We assume that $i_1, i_2, \dots, i_k$ be the videos in A^* which in front of i . Here is the total time of i and j in A^* and A' :

- i in A^* : $t_i = f_{i_1} + f_{i_2} + \dots + f_{i_k} + f_i + s_i$

- j in A*: $t_j = f_{i_1} + f_{i_2} + \dots + f_{i_k} + f_i + f_j) + s_j$
- i in A': $t'_i = f_{i_1} + f_{i_2} + \dots + f_{i_k} + f_j + f_i) + s_i$
- j in A': $t'_j = f_{i_1} + f_{i_2} + \dots + f_{i_k} + f_j) + s_j$

We find that $t'_j \leq t_j$ and $t'_i < t_j$ so that $\max(t'_i, t'_j) < \max(t_i, t_j)$. Since the A' has less or equal time than A*, A' is as optimal as A* but A' has fewer inversion than A*, contradict that S* has at least one inversion. Thus, A* has no inversion. Since A* has no inversion, all schedule has no inversion share the same maximum time and algorithm produce the schedule with no inversion.

QED.

Running time: the running time is $O(n \log n)$ because the algorithm is dominated by the sorting list.

2 Q2

The algorithm implements a greedy scheduling strategy to select tasks from a set $\mathcal{J}$ to maximize total profit. Initially, all time slots $S(i)$ are set to infinity, indicating that no tasks have been assigned, and a variable l is set to infinity to track the latest possible scheduling time. During the scheduling process, while $l > 1$ and the task set $\mathcal{J}$ is not empty, tasks that can be completed before time l and have deadlines $d_i \geq l$ are selected. Among these, the task j with the highest profit is chosen. Task j is then scheduled in the time slot $S(j)$ and the current time l is updated, with the scheduled task j removed from the task set. Finally, the ordered sequence of scheduled tasks $\mathcal{S}$ is returned. This approach ensures that the total profit is maximized within the given time constraints.

Algorithm 2 ORDER($\mathcal{J}$)

```

1: for  $i = 1$  to  $n$  do
2:    $S(i) := \infty$ 
3: end for
4:  $l := \infty$ 
5: while  $l > 1$  and  $\mathcal{J} \neq \emptyset$  do
6:    $l := \min(t, \max(d_i : \text{for every element in } \mathcal{J}))$ 
7:    $j :=$  the order of the maximum profit which  $d_j \geq t$ , if we find two order
      with same profit, choose the one with later deadline
8:    $S(j) := l$ 
9:    $\mathcal{J} := \mathcal{J} -$  the element in  $\mathcal{J}$  which order is  $j$ 
10: end while
11: return  $\mathcal{S}$ 

```

CORRECTNESS. The algorithm is correct because every time we choose the most profitable order and make it as late as possible to make the order list as tight as possible. I use promising set method to prove that it produces an optimal set.

We define S^* be one of the optimal schedule which schedules every order in S with same slot. We define the S^* extends S if for every order j in S , such that $S(j) \neq \infty, S^*(j) = S(j)$. We assume that S_t is the schedule after t iteration. It is obvious that S_t is a schedule which doesn't have some orders in same slot and schedule every order by their deadline.

Lemma 1: For every iteration t , there is an optimal set S_t^* extends S_t

Proof: By induction on the iteration t . The basis hold because $S_0 = \infty$ for every order i which means no order.

By induction step, suppose that there is an optimal schedule S_t^* for S_t . WTP: For iteration $t+1$, then there exist optimal set S_{t+1}^* extended by S_{t+1} . Suppose the order j is scheduled in iteration $t+1$ and l is the slot j will assign in. Then we consider two case:

- CASE 1: $S_t^*(j) = \infty$ which means j is not scheduled in iteration t . Then if the slot l is empty, we can schedule j into l to make more profit. However, this is contradict with that S^* is the optimal set. Thus, there must be some

j' is scheduled into l in S_t^* . Let S_{t+1}^* is identical to S_t^* except that instead of j' we schedule j in slot l : $S_{t+1}^*(j) = l = S_{t+1}(j)$ and $S_{t+1}^*(j') = \infty$. Note that when j is added to set in iteration $t+1$, j' is still in unscheduled set. In the meanwhile, $d_{j'} \geq l$ because that j' is scheduled in slot l in S_t^* . Because that $j' \in \mathcal{J}$ in iteration $t+1$, $d_{j'} \geq l$ and the algorithm add j rather j' ($p_j \geq p_{j'}$). The profit in S_{t+1}^* is greater or equal than S_t^* . S_{t+1}^* is at least as profitable with S_t^* , so S_{t+1}^* is optimal. By the definition and induction step, we hold statement, S_{t+1}^* is optimal and extends S_{t+1} as wanted

- CASE 2: $S_t^*(j) = l'$ for some $l' \neq \infty$. If $l' = l$, then we get $S_{t+1}^* = S_t^*$, and we are done. If $l' \neq l$, there are also two cases, depending on whether in S_t^* is scheduled in slot l .

- case 1: there is no such j' make $S_t^*(j') = l$. Then let S_{t+1}^* be same with S_t^* , except that S_{t+1}^* scheduled j in l instead of l' ; $S_{t+1}^*(j) = l = S_{t+1}(j)$, and the other is same with S_t^* . It is clear that S_{t+1}^* has same profit with S_t^* , so it is optimal. Furthermore, it extends S_{t+1} . By induction step, we hold the statement

- case 2: there exist such j' make $S_t^*(j') = l$. WTP: $l > l'$. (*) Prove by contradiction, assume $l' > l$. Before the $t+1$ iteration, order j is in $mathcal{J}$ and $d_j \geq l'$ (j is scheduled in slot l' in S_t^*). In this way, there are some order j'' scheduled in l' . Since S_t^* extends S_t , S_t^* also schedules j'' in l' . From the assumption in CASE 2, $j'' = j$, which is contradict with that we scheduled j in $l' \neq l$, then we proved (*). So S_t^* schedules order j' in slot l and order j in slot $l' < l$. Now let S_{t+1}^* same with S_t^* except that the slots of j and j' are swapped: $S_{t+1}^*(j) = l$ and $S_{t+1}^*(j') = l'$. It is valid because $d_j \geq l$ (since S_t schedules j in slot l) and $d_{j'} \geq l'$ (since S_t^* schedules j' in $l' > l$). S_{t+1}^* has same profit than S_t^* . Then we get proof, QED.

This algorithm will terminate because every iteration will remove one element in $\mathcal{J}$. Thus the maximum iteration is n . Assume the final result is S , then according to lemma, there must be a S^* extends S . We claim $S^* = S$. Suppose for contradiction, the $S^* \neq S$, there must exist j such that $S(j) = \infty$ and $S^*(j) = l \neq \infty$. If $S(j) = \infty$, then j will be considered every iteration because it is in $\mathcal{J}$ every time. According $S^*(j) = l, d_j \geq l$, order j should be able to schedule in l . Since j is always in $\mathcal{J}$, the only reason for there is no order be scheduled in l is that $d_j < l$, but is contradict with $S^*(j) = l$. Then we get $S^* = S$, which the result is optimal.

Running time: line 1 & 2 is $O(n)$. line 4 is $O(1)$. line 5 at most $O(n)$. line 6 & 7 is $O(n)$. line 8 & 9 is $O(1)$. The final complexity is that $O(n) + O(1) + O(n \times (n + n + 1 + 1)) = O(n^2)$

another proof: let's define something called Actual_profit: $ap(Is(I))$ which represent the profit of job I on time $s(I)$. from the question we know that the actual profit will equal to p_I if $s(I)$ is not infinite and it equals to 0 if $s(I)$ is infinite.

suppose the biggest deadline is big_d

then we can know that $\sum_1^n ap(is(i))$ which is equals the profit we earned fome time 1 to bid_d to will be equal to the sum of profit we can earn, which is the value that we want to maximized.

let's prove that through each iteration, we make the the sum of actual profit from time l to time big_d (i.e. sum of profit of job i such that $l \leq s(i) \leq big_d$) as big as possible. let's first prove that the job j which is we got from the algo is the job to make $ap(jt)$ biggest in $\mathcal{J}$. it is because if there is a job with higher profit, then its deadline should be before time t . through induction. for the first iteration, at time big_d we pick the job with biggest actual_profit, which is as wanted.

suppose for the K_{th} iteration $1 \leq k \leq n$ it holds.

for $k = n$, consider the job j as what the algo gives us, and we make $s(j) = t$, if there is a better choice to make the the sum of actual profit from time t to time big_d as big as possible, (make the sum bigger) then by comparing the sum from time $t+1$ to time big_d suppose that the job done at time t is j' in the better choice. because sum from time $t+1$ to time big_d in optimal solution is no bigger than what we get from the algo, which means the actual_profit of j' at time t will be bigger than j .

consider j' . $ap(j't)$ should be bigger than $ap(jt)$ because sum of actual profit from time $t+1$ to time big_d is biggest. and the j' should be found in the jobs that are done from time $t+1$ to time big_d by algo. and the algo did as much job as possible, so there must exist a job k which exist in the optimal choice but not in the algo, and let's consider its profit, should be no greater than job j or we will find the job k from time $t+1$ to time big_d by algo. and the so called optimal will not be optimal because sum made by algo $= p_j + others \geq p_k + others_2$ with n -knowing that $p_j \geq p_k$ and $others_1 \geq others_2$

base on the induction our IH is correct.

when the loop terminates, if time 0 has no jobs distributed, which means we did all the jobs and earn all the profit, we are correct. else, based on the induction, we earned as much profit as possible

2.1 Q3

The algorithm is designed to find a potential edge in a graph $\mathcal{G}$ that minimizes the total path weight from a source node s to a target node t . It begins by using Dijkstra's algorithm to calculate the shortest path weight $\delta(u)$ from the source s to each node u . Then, the algorithm reverses all graph edges and applies Dijkstra's algorithm again to determine the shortest path weight $\delta'(v)$ from each node v to the target t . For each potential edge (u, v) , it computes the sum $\delta(u) + \text{wt}(u, v) + \delta'(v)$, where $\text{wt}(u, v)$ is the weight of the edge. The algorithm keeps track of the edge that minimizes this sum and returns it as the optimal potential edge.

Algorithm 3 FIND($\mathcal{G}$)

- 1: For each u , using Dijkstra's algorithm to find the weight $\delta(u)$ to find the shortest-path from s to u .
 - 2: Reversing the edges, using Dijkstra's algorithm to find the weight $\delta'(v)$ of a shortest path from v to t , for each node v
 - 3: For each potential edge (u, v) , compute the sum $\delta(u) + \text{wt}(u, v) + \delta'(v)$ and find the minimum of the all the sum
 - 4: **return** the responding potential edge.
-

Running time: Each use of Dijkstra's algorithm in steps (1) and (2) takes $O((m+n)\log n)$ time. Reversing the edges in step 2 takes $O(m+n)$ time. So, steps (1) and (2) take $O((m+n)\log n)$ time. Step (3) takes $O(k)$ time, and step 4 takes $O(1)$ time. So, the algorithm takes $O(k + (m+n)\log n)$ time.

Correctness: Assume $e^* = (x, y)$ is one of the most optimal edge make the sum $E^* = \delta(x) + \text{wt}(x, y) + \delta'(y)$ as minimum. The result of the algorithm is $e' = (u, v)$, and the sum is E' .

WTP: the algorithm will return the $e' = (u, v)$ such that adding e' we can get the shortest path from s to t with only adding one potential edge.

To prove the statement, we should consider two cases:

- CASE 1: if adding e' does not affect the shortest path from s to t , In this case, it is obvious correct because as the problem mentioned that any potential edge can be correct.

- CASE 2: If adding the potential edge e' will affect the consequence.

let's first prove that the algorithm before line 3.5, the sum we get through $E^* = \delta(x) + \text{wt}(x, y) + \delta'(y)$ will be the shortest path from s to t via (x, y) without going on (x, y) twice (WTP2).

- SUB-CASE 1: the shortest path go through $s \rightarrow u \rightarrow v \rightarrow t$. According to the line 3, for the potential edge $e' = (u, v)$, we get the $\delta(u)$ (line 1), with $\delta'(v)$ (line 2), all of these are the minimum because the shortest-path. Calculating the sum of $\delta(u) + \text{wt}(u, v) + \delta'(v)$ keep track the minimum of them. so the algorithm will find the shortest path through $s \rightarrow u \rightarrow v \rightarrow t$. because $\min + \text{const} + \min$ will be the min distance .

- SUB-CASE 2: the shortest path go through $s \rightarrow v \rightarrow u \rightarrow t$. According to the line 3, for the potential edge $e' = (u, v)$, we get the $\delta(v)$ (line 1), with $\delta'(u)$

(line 2), all of these are the minimum because the shortest-path. Calculating the sum of $\delta(v) + wt(u, v) + \delta'(u)$ keep track the minimum of them. so the algorithm will find the shortest path through $s \rightarrow v \rightarrow u \rightarrow t$. because $\min + \text{const} + \min$ will be the min distance .

based on the two sub cases we prove the WTP2

and after the the line 3.5 we will get the path with the smallest distance among all the shortest paths through s to t via any potential paths (x,y).

so the algorithm can get the shortest path through s to t with adding only one potential path .

WTP is proved.

and back to the question, the algorithm should return the e' which can find the single potential edge in E' that, if added to the graph, reduces as much as possible the weight of a shortest $s \rightarrow t$ path, which is the same as finding the edge in all potential edges which can make the shortest path from s to t shortest . QED.